



# AFL++ Code Analysis

**20241900** 윤예원

# Table of Contents.

**001 AFL**

**002 AFL++**

**003 Seed selection**

**004 trim\_case()**

# AFL 동작 흐름

| P.03

- afl-fuzz()
  - init\_setting()
  - read\_testcase()
  - cull\_queue()
  - while(1)
    - 현재 시드 선택
    - cull\_queue()
    - fuzz\_one()
      - (조건 따라 skip 가능)
      - calibrate\_case()
      - trim\_case()
      - calculate\_score()
      - deterministic mutations
        - common\_fuzz\_stuff()
      - havoc\_stage
        - common\_fuzz\_stuff()
      - splicing\_stage
        - 다시 havoc\_stage
    - 다음 시드로 이동

## 전체 흐름

AFL은 seed queue를 반복적으로 순회하면서 입력을 퍼징함

실제 퍼징의 중심은 fuzz\_one()이며,  
이 안에서 calibration, trimming, score 계산, mutation이 수행됨

mutation 후 새로운 coverage가 발견되면  
queue에 새 seed가 추가되고, 이후 다음 시드로 넘어감

# AFL++ 구조: 기능별 분리

- ▼ AFLplusplus
  - ▼ src
    - C afl-analyze.c
    - C afl-cc.c
    - C afl-cmin.c
    - C afl-common.c
    - C afl-forkserver.c
    - C afl-fuzz-bitmap.c
    - C afl-fuzz-cmplog.c
    - C afl-fuzz-coverage.c
    - C afl-fuzz-extras.c
    - C afl-fuzz-frameshift.c
    - C afl-fuzz-ijon.c
    - C afl-fuzz-init.c
    - C afl-fuzz-mutators.c
    - C afl-fuzz-one.c
    - C afl-fuzz-python.c
    - C afl-fuzz-queue.c
    - C afl-fuzz-redqueen.c
    - C afl-fuzz-run.c
    - C afl-fuzz-sanfuzz.c
    - C afl-fuzz-skipdet.c
    - C afl-fuzz-state.c
    - C afl-fuzz-stats.c
    - C afl-fuzz-statsd.c
    - C afl-fuzz.c
    - C afl-gotcpu.c
    - C afl-ld-lto.c
    - C afl-performance.c
    - C afl-sharedmem.c
    - C afl-showmap.c
    - C afl-tmin.c
    - C hashmap.c

## 역할

전체 흐름 제어 / 시작점

seed 단위 퍼징

seed 선택 / queue 관리

target 실행

상태 관리

초기화

## AFL++ 파일

afl-fuzz.c

afl-fuzz-one.c

afl-fuzz-queue.c

afl-fuzz-run.c

afl-fuzz-state.c

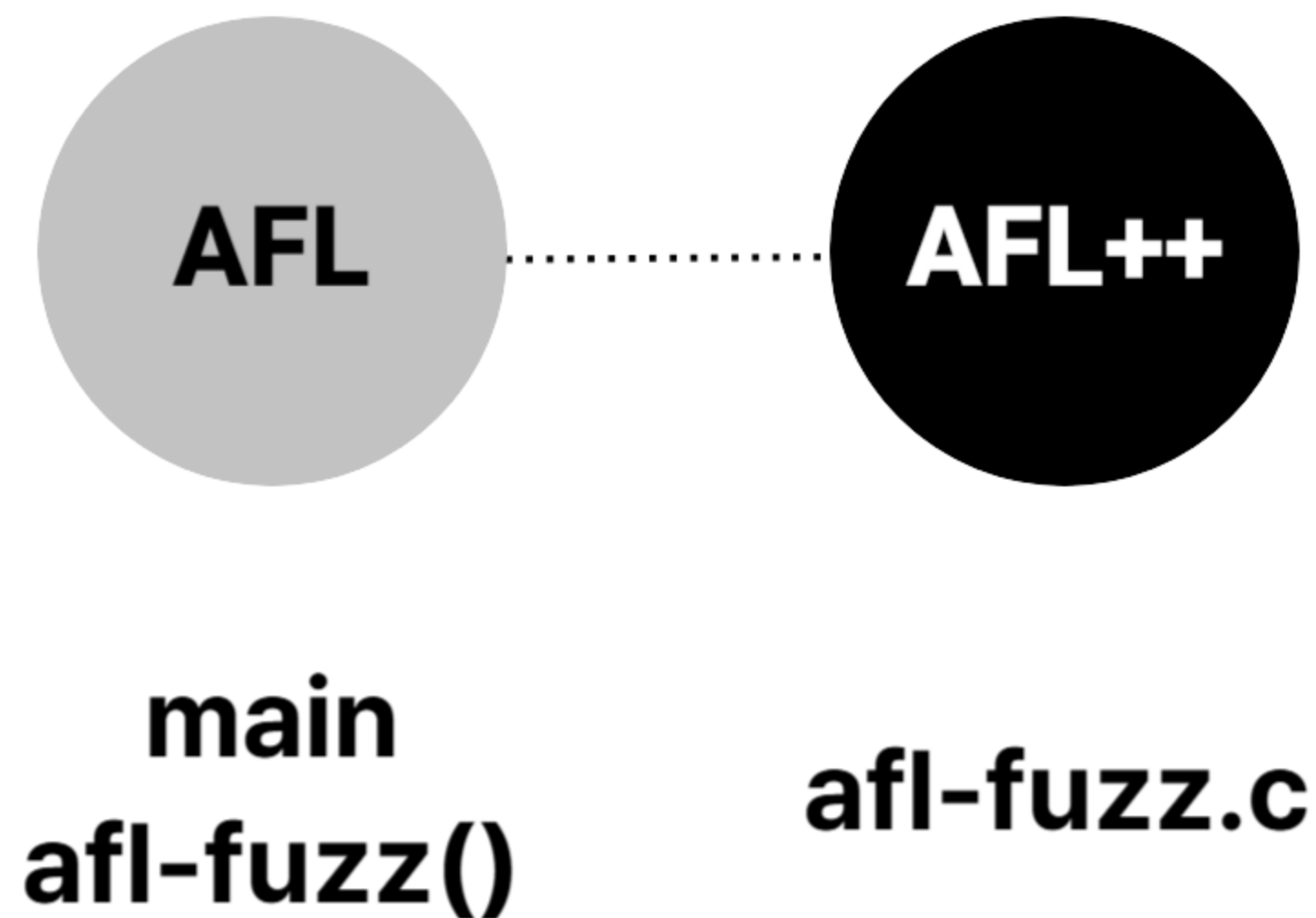
afl-fuzz-init.c

AFL 대비 구조 변화 (단일 파일 → 모듈화)

# AFL 흐름 -> AFL++ 파일 매핑

## afl-fuzz.c

```
598  ✓ int main(int argc, char **argv_orig, char **envp) {
599
600     s32 opt, auto_sync = 0 /*, user_set_cache = 0*/;
601     u64 prev_queued = 0;
602     u32 sync_interval_cnt = 0, seek_to = 0, show_help = 0, default_output =
603     |   map_size = get_map_size();
604     u8 *extras_dir[4];
605     u8 mem_limit_given = 0, exit_1 = 0, debug = 0,
606     |   extras_dir_cnt = 0 /*, have_p = 0*/;
607     char *afl_preload;
608     char *san_abstraction;
609     char *frida_afl_preload = NULL;
610     char **use_argv;
611
612     struct timeval tv;
613     struct timezone tz;
614
615     doc_path = access(DOC_PATH, F_OK) != 0 ? (u8 *)"docs" : (u8 *)DOC_PATH;
616
617  ✓ if (argc > 1 && strcmp(argv_orig[1], "--version") == 0) {
618
619     printf("afl-fuzz" VERSION "\n");
620     exit(0);
621
622 }
```



# AFL 흐름 -> AFL++ 파일 매핑

## afl-fuzz-queue.c

```
940 void cull_queue(afl_state_t *afl) {
941
942     if (likely(!afl->score_changed || afl->non_instrumented_mode)) { return
943
944     u32 len = (afl->fsrv.map_size >> 3);
945     u32 i;
946     u8 *temp_v = afl->map_tmp_buf;
947
948     afl->score_changed = 0;
949
950     memset(temp_v, 255, len);
```

```
680 void add_to_queue(afl_state_t *afl, u8 *fname, u32 len, u8 passed_det) {
681
682     struct queue_entry *q =
683         (struct queue_entry *)ck_alloc(sizeof(struct queue_entry));
684
685     q->fname = fname;
686     q->len = len;
687
688     q->depth = afl->cur_depth + 1;
689     q->passed_det = passed_det;
690     q->mother = afl->queue_cur;
```



AFL

cull\_queue()  
add\_to\_queue()



AFL++

afl-fuzz-queue.c

# AFL 흐름 -> AFL++ 파일 매핑

## afl-fuzz-one.c

```
328  u8 fuzz_one_original(afl_state_t *afl) {
329
330     u32 len, temp_len;
331     u32 j;
332     u32 i;
333     u8 *in_buf, *out_buf, *orig_in, *ex_tmp;
334     u64 havoc_queued = 0, orig_hit_cnt, new_hit_cnt = 0, prev_cksum, _prev_
335     u32 splice_cycle = 0, perf_score = 100, orig_perf;
336
337     u8 ret_val = 1, doing_det = 0;
338
339     u8  a_collect[MAX_AUTO_EXTRA];
340     u32 a_len = 0;
341
342     /* IJON: If we're doing IJON, skip deterministic stages and go directly
343     * havoc */
344     if (unlikely(afl->is_doing_ion)) {
345
346         /* Use IJON input data that was set up in fuzz_one() */
347         len = afl->ijon_input_len;
348         orig_in = in_buf = afl->in_buf;
349         out_buf = afl->out_buf;
350         memcpy(in_buf, afl->ijon_input_data, len);
351         memcpy(out_buf, afl->ijon_input_data, len);
352
353         /* Setup variables for havoc stage */
354         temp_len = len;
355         orig_hit_cnt = afl->queued_items + afl->saved_crashes;
356         havoc_queued = afl->queued_items;
357         perf_score = 100;
358         orig_perf = perf_score;
359
360         /* Jump directly to havoc stage */
361         goto havoc_stage;
362     }
363 }
```



AFL

fuzz\_one.c



AFL++

afl-fuzz-one.c

# AFL 흐름 -> AFL++ 파일 매핑

| 기능             | AFL               | AFL++            |
|----------------|-------------------|------------------|
| 전체 흐름 제어 / 시작점 | main, afl-fuzz()  | afl-fuzz.c       |
| 큐 정리           | cull_queue        | afl-fuzz-queue.c |
| 한 시드 퍼징        | fuzz_one()        | afl-fuzz-one.c   |
| target 실행      | run_target()      | afl-fuzz-run.c   |
| 보정 / 안정성 확인    | calibrate_case()  | afl-fuzz-one.c   |
| 트리밍            | trim_case()       | afl-fuzz-one.c   |
| 점수 계산          | calculate_score() | afl-fuzz-queue.c |
| 새 시드 추가        | add_to_queue()    | afl-fuzz-queue.c |

# Seed Selection: AFL vs AFL++

**AFL**

`queue_cur = queue_cur → next`

queue를 순차적으로 순회

favored / skip 조건으로 일부 조정

**AFL++**

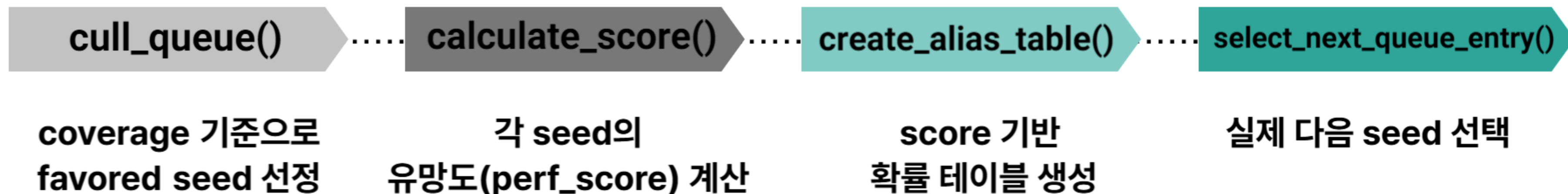
`select_next_queue_entry()`

alias table 기반  
weighted random selection

`calculate_score()`와 연결

AFL은 순차 선택, AFL++는 score 기반 확률 선택

# AFL++의 Seed Selection 내부 구조



# Favored Seed Selection: cull queue()



favored는 "좋아 보이는 seed"가 아니라 coverage를 대표하는 seed

# Score & Probability Selection in AFL++

| P.12

## score 계산

`calculate_score()`

각 seed에 perf\_score 부여

반영 요소 : execution time  
coverage  
favored 여부

## 확률 생성

`create_alias_table()`

perf\_score 기반 확률 테이블 생성

alias table / alias probability 저장

→ 빠른 weighted random selection 위해

perf\_score  
↓  
확률 분포 생성  
↓  
alias table 구성



# Seed Selection: `select_next_queue_entry()`

```
48 inline u32 select_next_queue_entry(afl_state_t *afl) {  
49  
50     u32    s = rand_below(afl, afl->queued_items);  
51     double p = rand_next_percent(afl);  
52  
53     return (p < afl->alias_probability[s] ? s : afl->alias_table[s]);  
54  
55 }
```

# Input Minimization: trim\_case()

| P.14

## 목적

입력 크기 감소  
불필요한 부분 제거  
이후 mutation 효율 향상

## 동작 흐름

입력 일부 제거 시도



run\_target()



coverage 동일?



YES → 유지

NO → 복구

# trim\_case(): AFL vs AFL++

## AFL

기본 trimming 알고리즘

입력 일부 제거 후 실행

coverage 동일 시 삭제 유지

상대적으로 단순한 구조

## AFL++

custom mutator 기반 trimming 지원

testcase 재동기화

afl\_state\_t 기반 상태 관리

trim 조건이 더 명시적임

AFL++는 AFL의 trimming 아이디어를 유지하면서,  
더 확장 가능하고 구조화된 형태로 발전시킴

**AFL : 단순 순차 탐색**  
**AFL++ : score 기반 확률 선택**